

# Carbu-Rat

## Rapido & Razmo, the fuel-price rats

A fuel-price assistant and threshold alerts on Telegram, built on n8n

**Josh E. SOUSSAN**

October 2026 · Level 3



n8n · Telegram ×2 · NanoGPT · n8n Data Tables · Fuel-price open data · French geocoding API

# The need

**15–20 c**

more per litre

depending on the station you pick, in the same area, on the same day

**Drivers in France check fuel prices on a comparison website by hand, when they remember to, and never know when prices drop.**

- Prices change **every day**.

**Goal:** answer “where do I fill up?” in seconds, and warn me *once* when my target price is reached.

# Meet the duo: two rats, two bots



## Rapido

@rapido_carbu_bot

**the one you ask**

- `/plein Créteil gazole`: top 5 cheapest pumps nearby
- Free text, answered by an **AI agent**
- Small, fast, short sentences
- **Pull**: only ever answers



## Razmo

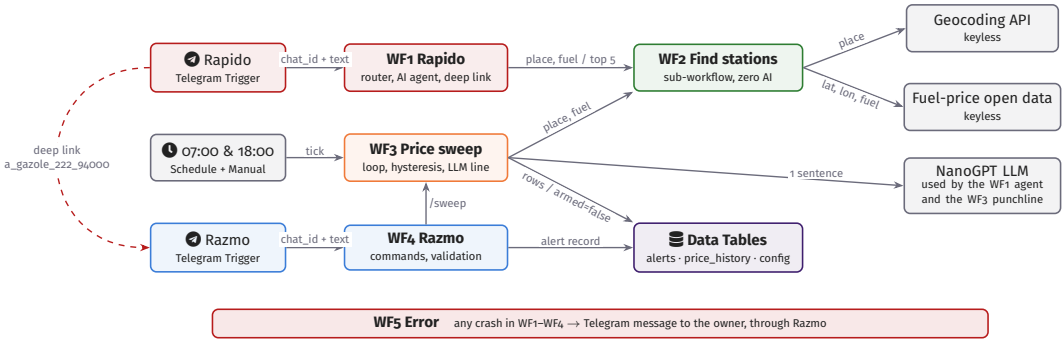
@razmo_carbu_bot

**the one who watches**

- `/alerte gazole 1,65 Créteil`
- Checks the pumps **twice a day**
- Big, calm, speaks of himself in the 3rd person
- **Push**: the only one who writes first

Two bots, so the user can **mute the alerts without losing the assistant**.
A nod to the cartoon *Ratz*; the lines are our own.

# Architecture: five workflows, one journey




**Level 3.** Cut line, first dropped: `/tendance` → deep link → LLM punchline → agent memory. Never cut: `/plein`, `/alerte`, sweep + hysteresis, error workflow.

# Hysteresis: never the same ping twice




Each alert has an `armed` flag.

- **fire** if price $\leq$ threshold **and** armed
  then armed = false
- **re-arm** if price $>$ threshold + 2 c
  and the alert is disarmed
- otherwise **skip**

The 2-cent margin works like a thermostat: a price wobbling around the threshold does **not** spam the user.

# Live demo

1. **Rapido**: `/plein Créteil gazole` — round trip: top 5 + deep link
2. Tap the link — Razmo stores the alert
3. **Razmo**: `/alerte gazole 3,00 Créteil`, twice — one row only (upsert)
4. **Razmo**: `/sweep` — **1 ping**: state read in another run
5. **Razmo**: `/sweep` — **0 ping**, hysteresis
6. **Razmo**: `/panne` — bad URL → WF5 alert + run link
7. n8n **Executions** tab — green runs, one red drill



# Thank you!

Questions?

5 workflows · 85 nodes · 3 credentials · 2 rats

github.com/joshsssn/No-Code-Final-Project

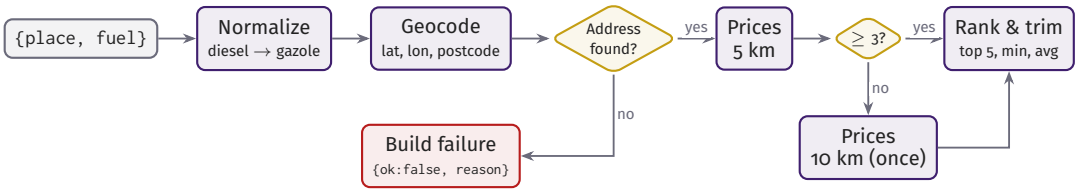




# Backup slides

for the questions

# WF2 · Find stations: the shared core (zero AI)




- **3 callers, written once**: `/plein`, the AI agent (as a tool) and the sweep.
- The radius widens **once** to 10 km when fewer than 3 stations are found.
- Field names were **checked with real API calls** before building the nodes.

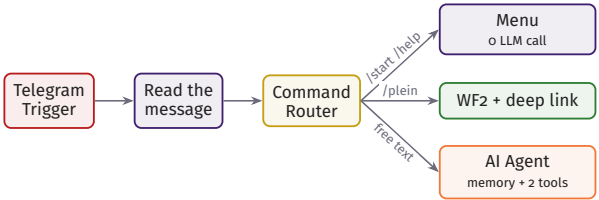
## Contract

`{ok:true, place_label, postcode, stations:[top 5], min, avg, spread, radius_km}`

`{ok:false, reason: place_not_found | unknown_fuel | api_down}`

A few hundred bytes, whatever the API returns.

# WF1 · Rapido: where to fill up




- **AI agent**: NanoGPT model, memory of 10 messages per user.
- Tools: `find_stations` (WF2) and `my_alerts` (alerts table, filtered on the user's `chat_id` by the workflow, not by the LLM).
- All branches end in **one** Send node: HTML, split under 4,000 characters.

## Hand-over to Razmo

Every `/plein` answer ends with a pre-filled deep link:

```
t.me/razmo_carbu_bot?start=
   a_gazole_222_94000
```

| | |
|---|---|
| `gazole` | fuel |
| `222` | threshold in cents: best price − 3 c |
| `94000` | postcode |

Telegram only allows `[A-Za-z0-9_-]` in the payload, hence cents and underscores.

# WF4 · Razmo: price thresholds

| Command | What Razmo does |
|---|---|
| `/alerte gazole 1,65 Créteil` | validates, then stores it |
| `/start a_gazole_165_94000` | same, from Rapido's link |
| `/mesalertes` | numbered list of alerts |
| `/stop 2` | deactivates alert n°2 |
| `/tendance` | lowest price per day, 7 days |
| `/sweep` | runs WF3 now (**owner**) |
| `/start` | menu; first user = owner |

**Valid alert?** (If node)

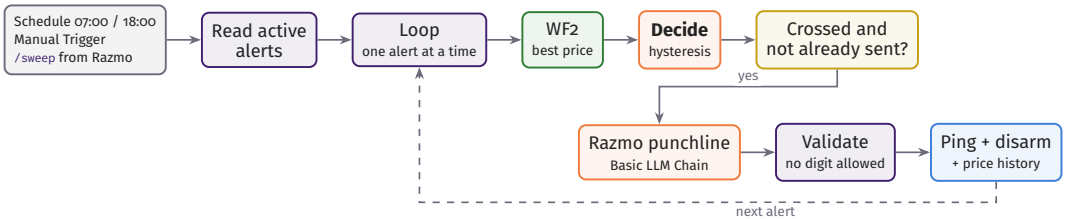
- ✔ known fuel (synonyms normalised)
- ✔ threshold between 0.50 and 3.50 €
- ✔ the place geocodes

French decimals accepted: `1,65` → 1.65

Otherwise Razmo explains *exactly* what is wrong, in character.

> *"Razmo a compris. Gazole sous 1,65 € à Créteil. Razmo bouge pas de la pompe."*

# WF3 · Price sweep




- One failing alert never blocks the others.
- `price_history`: one row per place, fuel and am/pm slot (upsert), which feeds `/tendance`.
- An alert is disarmed even if Telegram rejects the ping, so it never loops.
- `/sweep` replies with “N alerts checked, P pings sent”.

# AI with guardrails: the LLM decides the tone, never the price



## Where the AI is

- **Rapido**'s **AI Agent**: free-text questions, 2 tools, memory
- **Razmo**'s **punchline**: one sentence per ping

*Real punchline from the tests: "Razmo a vu que le gazole devient plus doux pour la bourse et ça lui fait plaisir."*



## Guardrails

- Every price shown is formatted by a **Code node** from API data
- The punchline is **rejected if it contains a digit** → one of 10 hand-written lines
- Agent prompt: "never a price that does not come from a tool"



## Provider

- NanoGPT, **OpenAI-compatible**: standard OpenAI Chat Model node, only the base URL changes
- Tool calling checked with a real call before building the agent
- Swapping provider = editing **one credential**

# Robustness: both rats keep answering, in character

| When… | …the system |
|---|---|
| an API is slow or down | retries 3 times, 2 s apart, then: *“Les pompes répondent pas à Razmo. Razmo patient.”* |
| fewer than 3 stations at 5 km | widens the search to 10 km, once |
| the place or fuel is unknown | explains the problem and shows the right syntax |
| the LLM fails | agent: falls back to `/plein`, which needs no AI; punchline: template line |
| the LLM writes a digit | rejects the sentence and uses a template line |
| a message is too long | splits it under 4,000 characters (Telegram limit) |
| the sweep runs twice | hysteresis: no duplicate ping; history upsert: no duplicate row |
| the same `/alerte` twice | upsert on (chat, fuel, postcode): still one alert |
| anything crashes | **WF5** sends the owner the workflow, the node and a link to the run (live test: `/panne`) |

# Stack and credential budget

## Only 3 credentials

| | |
|---|---|
| Telegram Rapido | 1 token, @BotFather |
| Telegram Razmo | 1 token, same chat |
| NanoGPT | 1 API key |
| Fuel-price open data | keyless |
| French geocoding API | keyless |
| n8n Data Tables | built in |

Avoided on purpose: Google Sheets (OAuth consent screen), hosted Postgres, paid fuel APIs.

## Runtime

- n8n 2.44.5 in **Docker Compose**, pinned images, named volume, healthcheck, encryption key from `.env`
- Telegram needs a public HTTPS URL: a free **Cloudflare tunnel** in a second container
- One script, `start-carbu.sh`: starts the tunnel, reads its URL, starts n8n, re-registers both Telegram webhooks
- Same `chat_id` in both bots: Rapido can read the alerts stored by Razmo

# Tested before the demo

- ✔ WF2: real place, nonsense place, synonym (`diesel`), unknown fuel
- ✔ Hysteresis: test alert at 3.00 €, two sweeps → fired once, then skipped
- ✔ Both parsers on 7 reference inputs, run on the deployed code
- ✔ Every Razmo route and both Rapido routes, end to end
- ✔ The agent calls `find_stations(Lyon, gazole)`; its prices match the tool output
- ✔ Telegram webhooks point to the tunnel, 0 errors

**Bug found, guardrail confirmed.** On the first agent test, the tool failed (“workflow is not active”: a sub-workflow used as an agent tool must be published). The agent did *not* invent a price; it answered “the tool does not respond, try /plein”. WF2 was published and the test passed.

# Known limits and next steps

## Known limits

- Agent memory lives in RAM: lost on restart
- Free tunnel: the URL changes on restart
- Owner = first `/start`; anyone can message the bots
- No brand names in the open data

## Next steps

- Group identical alerts: one API call per place and fuel
- Telegram location: “pumps around me”
- Persistent memory, allow-listed users
- Fixed domain instead of a quick tunnel

